DEEP LEARNING · COMPUTER VISION · EXPERIMENT REPORT

计算机视觉深度学习四项目完整实验报告

从目标检测到图像检索，按由易到难顺序完成 4 个 CV 深度学习项目 —— 包含**全部源代码、训练日志、评估指标与可视化结果**，实验平台为单卡消费级 GPU（RTX 4060 Laptop 8GB）。

日期 **2026-10-12** | PyTorch **2.11.0+cu128** | ultralytics **8.4.175** | GPU **RTX 4060 Laptop 8GB** | Python **3.13.0** | 实验脚本 **10 个**

目录

1. 实验总览与技术路线
2. 项目一：YOLOv8 目标检测（迁移学习对照实验）
3. 项目二：GAN 与 Diffusion 图像生成
4. 项目三：人脸关键点回归 + 表情识别
5. 项目四：CNN 特征图像检索
6. 总结：关键问题、修复与原理串联
7. 附录：全部项目源代码

1. 实验总览与技术路线

四个项目覆盖**检测、生成、回归/分类、检索**四大 CV 任务范式，难度递进，共享同一套 PyTorch 训练/评估方法论。

PROJECT 01 · 检测
0.816
YOLOv8s 优化版 v2 mAP50
（v1 基线 0.6664 / 从零仅

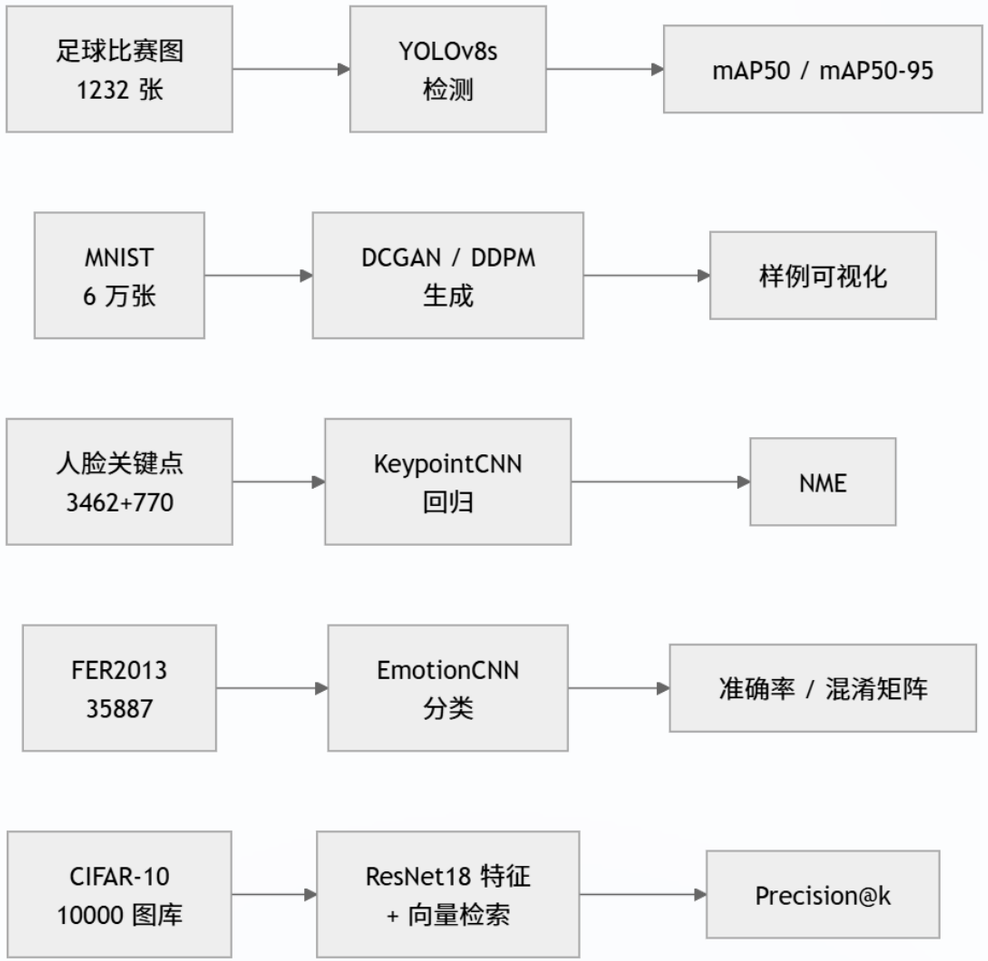
PROJECT 02 · 生成
30+20
DCGAN 30 轮 / DDPM 20 轮
生成清晰手写数字

PROJECT 03 · 回归
0.107
68 关键点 NME 误差
FER2013 表情 acc 65.1%

PROJECT 04 · 检索
74.8%
Precision@1（随机基线 9.8%）
CIFAR-10 万级图库

0.5321)

图 1·技术路线：数据集 → 模型 → 评估指标（四项目由易到难）



四个任务共享「数据准备 → 训练 → 评估 → 可视化」的标准实验流程

项目	任务	模型	数据集	核心指标	难度
01 目标检测	定位+分类	YOLOv8s（v2 优化版）	football (COCO, 1232 图)	`mAP50 = 0.816`	⭐⭐
02 图像生成	无监督生成	DCGAN / DDPM	MNIST (60000)	生成样本人眼可辨认	⭐⭐⭐

03 关键点	回归	KeypointCNN	68 点人脸 (3462)	NME = 0.107	⭐⭐⭐
03 表情	7 类分类	EmotionCNN	FER2013 (35887)	acc = 65.1%	⭐⭐⭐
04 图像检索	相似度检索	ResNet18-512d	CIFAR-10 (10000)	P@1 = 74.8%	⭐⭐⭐

2. 项目一：YOLOv8 目标检测

在足球比赛数据集上做**零样本推理 → 迁移学习微调 → 从零训练对照**的完整检测实验，量化预训练权重的价值。

2.1 数据准备

数据集为 COCO 格式（类别 `player` / `football`），脚本完成解压、**COCO bbox → YOLO 归一化 txt** 转换与 `data.yaml` 生成：

```
[开始] 足球检测数据集转换 (COCO → YOLO)
  [train] 859 张图, 类别: ['football', 'player']
  [valid] 243 张图, 类别: ['football', 'player']
  [test] 130 张图, 类别: ['football', 'player']
[样例] 8 张 → f:\deeplearn\projects\01_yolov8_detection\samples
[完成] YOLO 数据集 → f:\deeplearn\projects\01_yolov8_detection\dataset
       配置文件 → f:\deeplearn\projects\01_yolov8_detection\dataset\data.yaml
```

2.2 零样本推理（COCO 预训练权重）

直接用 `yolov8n.pt`（COCO 80 类预训练）推理 8 张样例图，检出 12~18 个目标/图，主要为 `person`：



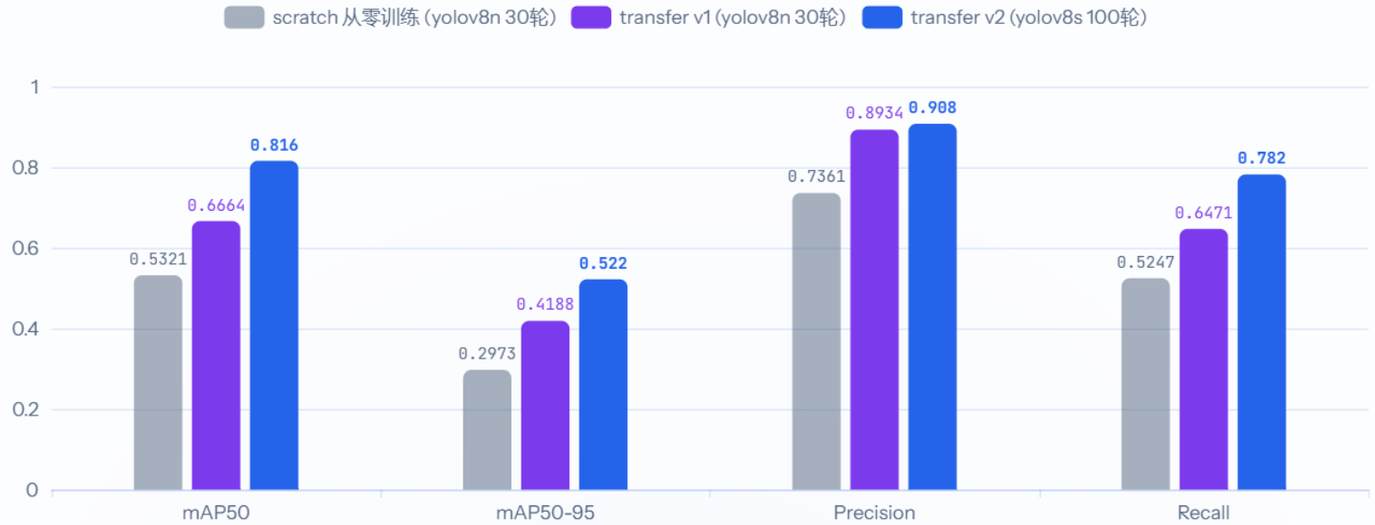


图 2·零样本推理结果样例（sample_06：15 个目标，最高置信度 0.72）

2.3 对照实验：从零训练 vs 迁移学习

v1 基线实验：同一数据集、同一超参数（imgsz=640，epochs=30，batch=16，lr0=0.01，patience=15），唯一变量是**初始化权重**。v2 优化版进一步升级：yolov8n→yolov8s、imgsz 640→832、epochs 30→100，并新增数据增强（batch=12，lr0=0.01，patience=30，device=0，mosaic=1.0，mixup=0.1，fliplr=0.5，close_mosaic=15，cache=True）：

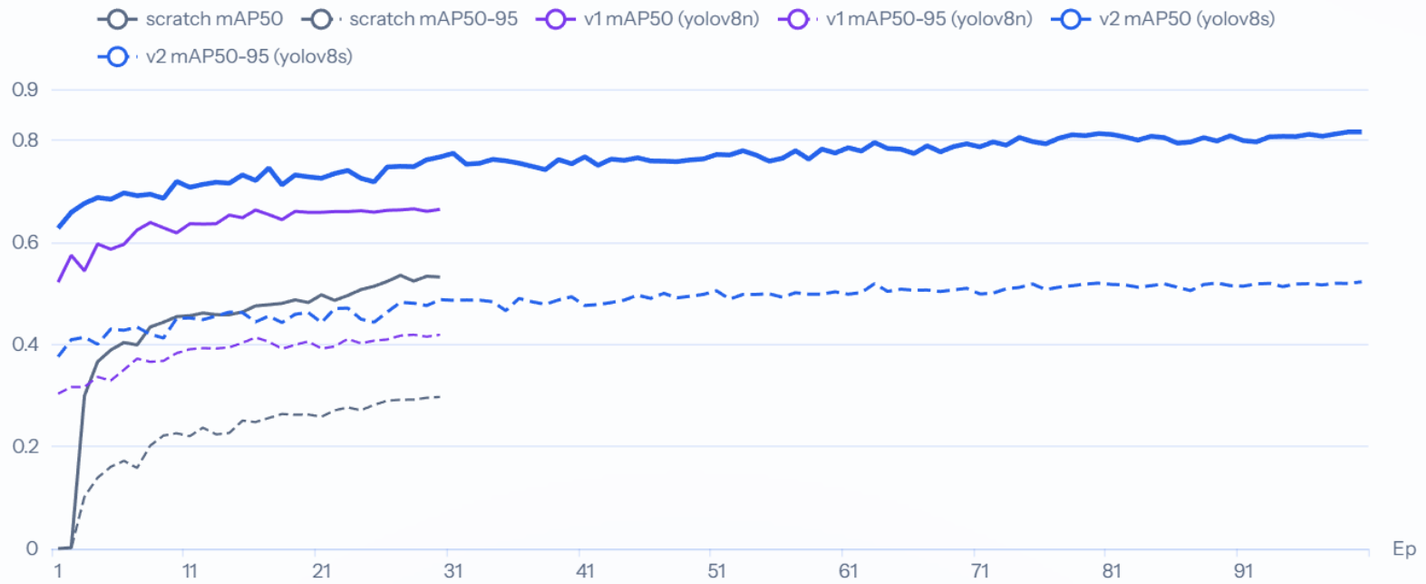
图 3·最终验证指标对比：scratch（从零）vs transfer v1（yolov8n 30 轮）vs v2 优化版（yolov8s 100 轮）



指标	scratch 从零训练 (yolov8n, 30 轮)	transfer v1 基线 (yolov8n, 30 轮)	transfer v2 优化版 (yolov8s, 100 轮)	v2 vs v1

mAP50 (IoU=0.5)	0.5321	0.6664	**0.816**	**+14.96 pp**
mAP50-95 (严格)	0.2973	0.4188	**0.522**	**+10.32 pp**
Precision	0.7361	0.8934	**0.908**	**+1.46 pp**
Recall	0.5247	0.6471	**0.782**	**+13.49 pp**
分类别 mAP50（football / player）	–	–	**0.643 / 0.990**	–
训练用时	8.8 分钟	7.7 分钟	52.5 分钟	+44.8 分钟

图 4 · 训练 mAP50 / mAP50-95 学习曲线（scratch / v1 为 30 轮，v2 优化版为 100 轮，数据源自各变体的 results.csv）



结论：迁移学习第 1 轮即达到 mAP50=0.52（接近 scratch 训练 30 轮的终点），v1 最终领先 scratch **12~16 个百分点**；v2 优化版（yolov8s + 100 轮 + imgsz 832 + 数据增强）进一步把 mAP50 提到 **0.816**（相对 v1 **+14.96 pp**、相对从零训练 **+28.4 pp**），player 类 mAP50 达 0.990，football 小目标仍是主要瓶颈（0.643）——小数据集上**预训练权重 + 模型容量 + 充分训练**的收益非常显著[1]。

2.4 训练日志（摘录）

v1 基线对照（yolov8n，30 轮）：

```
===== 实验 [scratch] 初始权重: yolov8n.yaml =====
engine\trainer: ... batch=16, epochs=30, imgsz=640, lr0=0.01, patience=15, device=0 ...
AMP: running Automatic Mixed Precision (AMP) checks...
[scratch] 完成，用时 8.8 分钟

===== 实验 [transfer] 初始权重: ...\weights\yolov8n.pt =====
engine\trainer: ... model=f:\deeplearn\projects\01_yolov8_detection\weights\yolov8n.pt ...
Transferred 349/355 items from pretrained weights
[transfer] 完成，用时 7.7 分钟

============================================================
指标                          scratch 从零训练       transfer 迁移学习
============================================================
mAP50 (IoU=0.5)                  0.5321             0.6664
mAP50-95 (严格)                   0.2973              0.4188
Precision                        0.7361             0.8934
Recall                           0.5247             0.6471
============================================================
迁移学习 mAP50 提升: +13.43 个百分点
```

v2 优化版训练（yolov8s，100 轮，2026-10-12 22:56 → 23:49）：

```
===== 优化版 v2 实验 [transfer] 初始权重: ...\weights\yolov8s.pt =====
engine\trainer: model=...\weights\yolov8s.pt, data=...\dataset\data.yaml, imgsz=832, epochs=100,
  batch=12, lr0=0.01, patience=30, device=0, mosaic=1.0, mixup=0.1, fliplr=0.5,
  close_mosaic=15, cache=True, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, scale=0.5, translate=0.1 ...
Overriding model.yaml nc=80 with nc=2
Model summary: 129 layers, 11,136,374 parameters, 11,136,358 gradients, 28.6 GFLOPs
Transferred 349/355 items from pretrained weights
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and determining best 'optimizer' and 'lr0' automatically...
optimizer: AdamW(lr=0.001667, momentum=0.9) with parameter groups 57 weight(decay=0.0),
           64 weight(decay=0.00046875), 63 bias(decay=0.0)

---- 最后 15 轮（epoch 86 ~ 100）逐轮验证指标（val=all，243 图 / 4513 实例）----
epoch     P       R     mAP50  mAP50-95
   86   0.868  0.775  0.794   0.512
   87   0.912  0.757  0.796   0.505
   88   0.917  0.766  0.805   0.517
   89   0.928  0.752  0.799   0.520
   90   0.939  0.769  0.809   0.515
   91   0.921  0.767  0.799   0.514
   92   0.885  0.770  0.797   0.519
   93   0.904  0.776  0.807   0.520
   94   0.875  0.781  0.808   0.513
   95   0.879  0.782  0.807   0.518
   96   0.889  0.789  0.812   0.519
```

```
  97   0.905  0.784  0.808   0.517
  98   0.901  0.786  0.812   0.520
  99   0.901  0.792  0.816   0.520
 100   0.906  0.782  0.816   0.523
epoch 100 训练损失: box=0.76693  cls=0.34980  dfl=0.79666
epoch 100 验证损失: box=0.86040  cls=0.38335  dfl=0.82503

100 epochs completed in 0.863 hours.
Optimizer stripped from F:\deeplearn\projects\01_yolov8_detection\results\transfer\weights\best.pt, 22.5MB

Validating F:\deeplearn\projects\01_yolov8_detection\results\transfer\weights\best.pt...
Ultralytics 8.4.175  Python-3.13.0 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 4060 Laptop GPU, 8188MiB)
Model summary (fused): 72 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)
                   all        243       4513      0.908      0.782      0.816      0.522
                   football   199        262      0.842       0.58      0.643      0.291
                   player     242       4251      0.975      0.984       0.99      0.753
Speed: 0.4ms preprocess, 3.8ms inference, 0.0ms loss, 1.8ms postprocess per image
Results saved to F:\deeplearn\projects\01_yolov8_detection\results\transfer
[transfer] 完成，用时 52.5 分钟
```

2.5 训练过程可视化

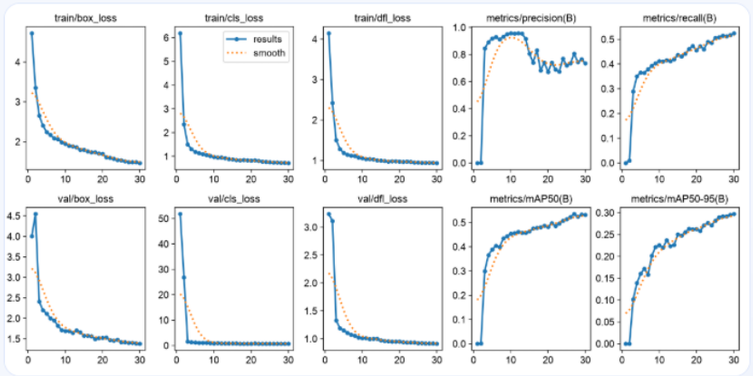


图 5·scratch 训练曲线（box/cls/dfl 损失 + P/R/mAP）

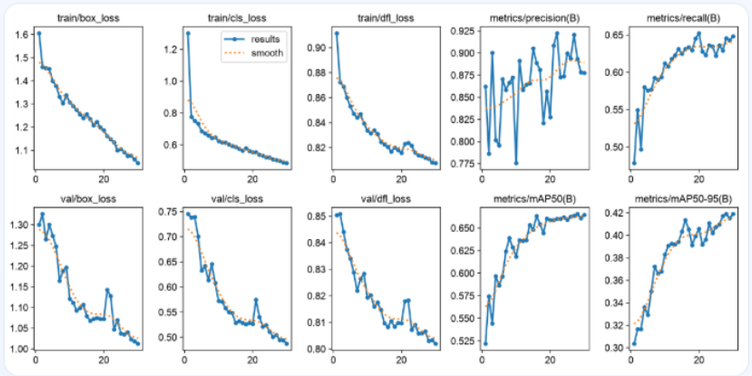


图 6·transfer 训练曲线（起点即接近收敛）



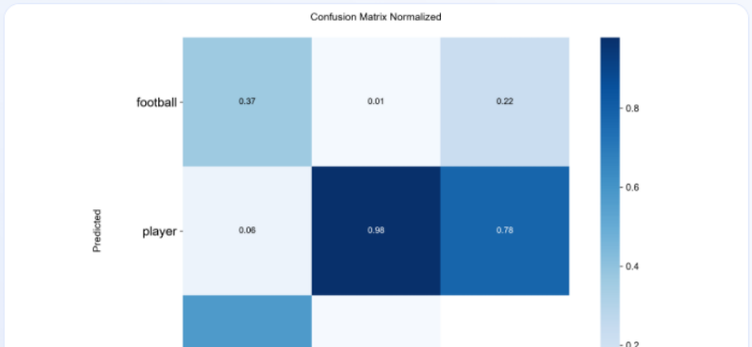




图 7·迁移学习模型验证集预测（绿框=预测）

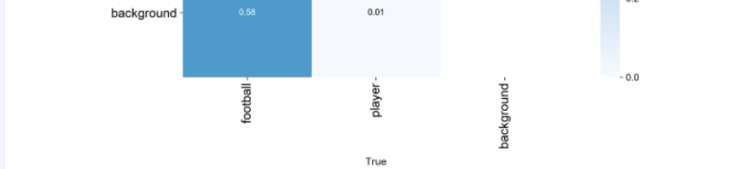


图 8·迁移学习归一化混淆矩阵

3. 项目二：GAN 与 Diffusion 图像生成

实现**两种生成范式**：DCGAN（对抗博弈）与 DDPM 扩散模型（逐步加噪-去噪），均在 MNIST 上训练。

3.1 DCGAN 对抗生成（30 轮）

生成器：全连接 → 4×上采样转置卷积（BatchNorm + ReLU，输出 Tanh）；判别器：4×步长卷积（LeakyReLU），最终 3×3 卷积压到 1×1 输出真假分数。训练中修复了三个结构性 bug：线性层未 reshape 成 4D、判别器卷核大于输入、多层上采样导致输出 56×56。

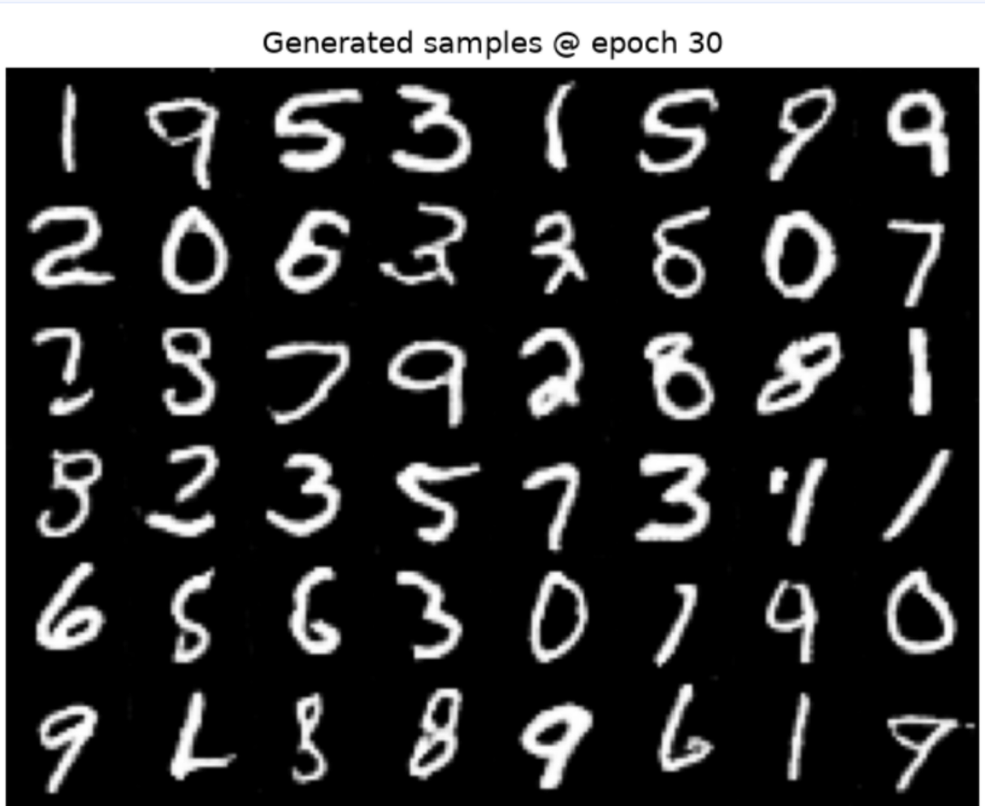




图 9 · DCGAN 第 30 轮生成样本网格（数字笔画清晰、风格多样）

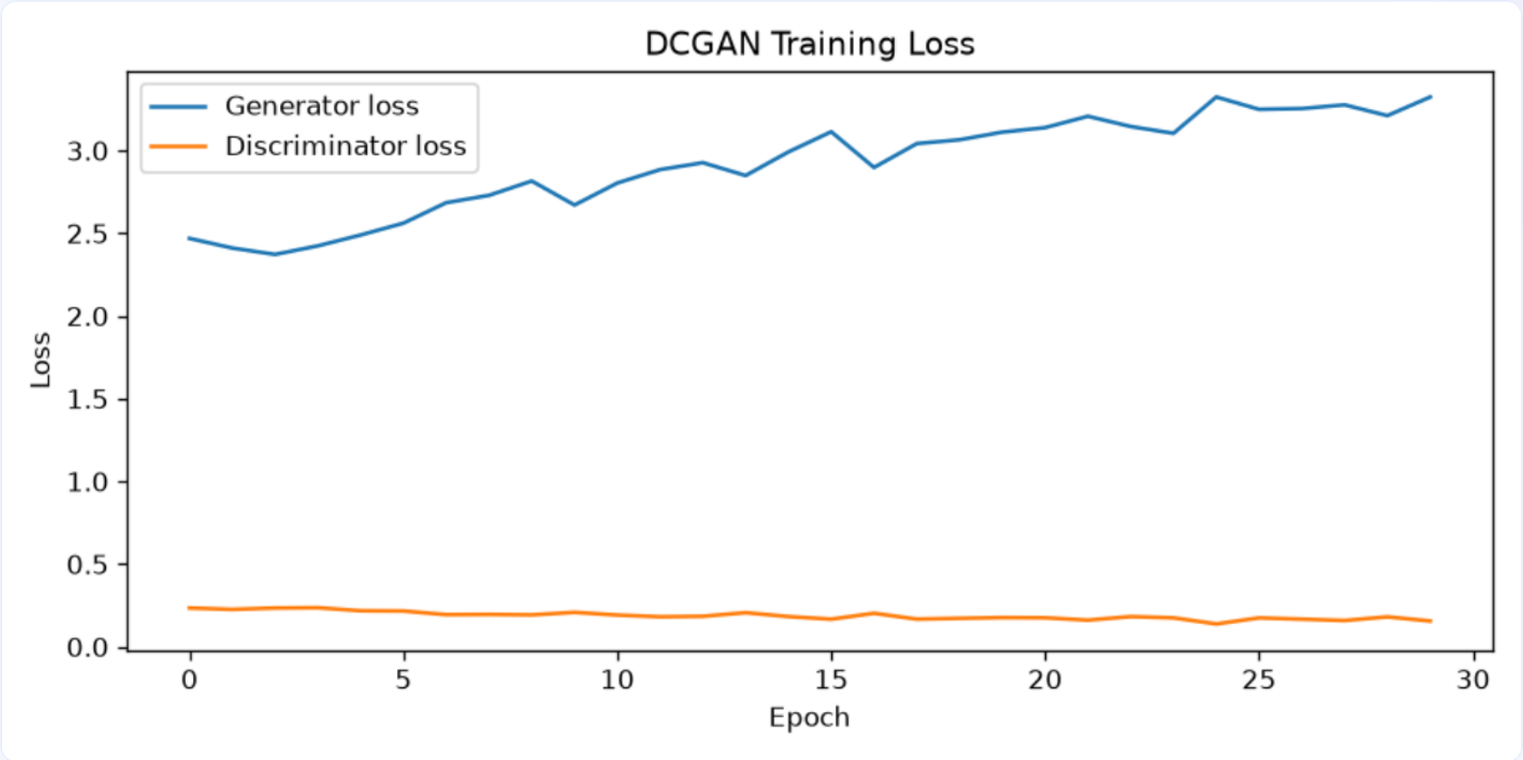


图 10 · GAN 对抗训练损失曲线（G/D 动态博弈趋于稳定）

3.2 DDPM 扩散生成（T=300，20 轮）

前向过程 `q_sample` 逐步加噪（余弦/线性 beta 调度）；**反向过程**用 MiniUNet 预测每步噪声，按后验公式逐步去噪采样。训练中修复了 `sample()` 中占位死代码（补齐 `alphas_bar` 与均值公式）。

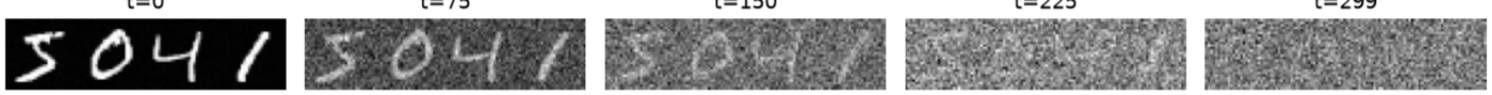


图 11·前向加噪链可视化（清晰图 → 纯高斯噪声）

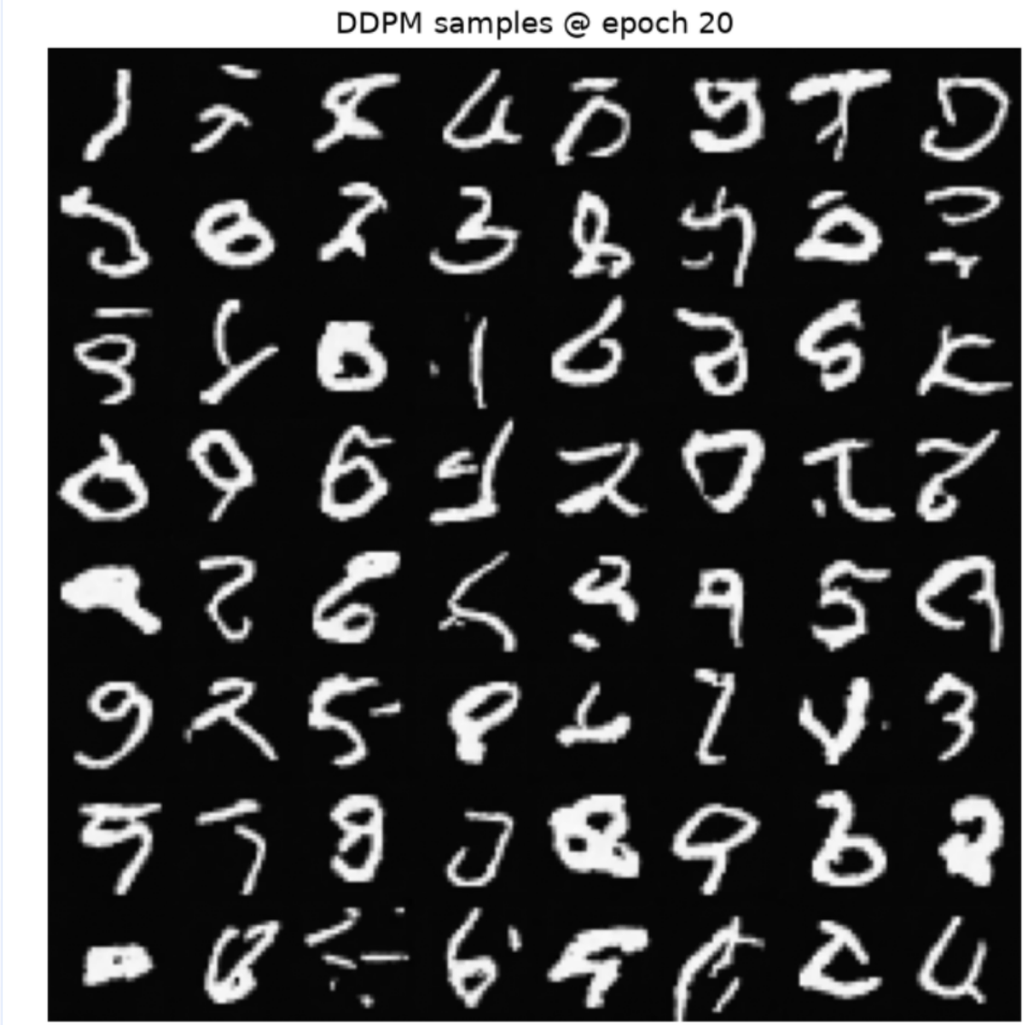


图 12·DDPM 第 20 轮反向去噪生成样本（数字基本成形，笔画略粗于 GAN）

对照观察：同一数据集上，DCGAN 30 轮生成的数字更锐利；DDPM 20 轮已可辨认但边缘较糊——DDPM 需要更多轮次与更大容量才能发挥

质量优势，但其训练稳定性（单一回归损失）优于 GAN 的对抗平衡[2][3]。

日志说明：本项目训练于早期会话，终端滚动日志已被系统清理；训练过程与轮次记录完整保存于上方三张曲线/网格图（`results/gen_epochXXX.png`、`ddpm_epochXXX.png`、`loss_curve.png`），最终模型权重为 `dcgan_G.pth` / `dcgan_D.pth` / `ddpm_unet.pth`。

4. 项目三：人脸关键点回归 + 表情识别

两个小数据集任务：68 个人脸关键点**坐标回归**与 FER2013 **七类表情分类**。

4.1 数据修复：逐图动态坐标缩放

本项目最关键的问题：原始数据集**每张图片尺寸都不同**（312×270、157×196、176×192…），68 点坐标按各自原图尺寸标注。最初按固定 176×192 缩放导致 y 值超出 [0,96]（最大 206），关键点全部错位。修复为 `sx = 96 / 该图宽度`，`sy = 96 / 该图高度` 逐图计算后，坐标恢复到 [-9, 97] 的合理范围（仅 19 个边缘点因人脸贴边轻微越界）。

4.2 68 关键点回归（KeypointCNN）

4 个卷积块（96→48→24→12→6）+ 全连接头输出 136 维（68×2），**掩码 MSE** 跳过缺失标注点，评估用 **NME**（归一化平均误差，除以双眼间距）。修复了 `nme()` 中 136→68 维掩码分组错误后完成 60 轮训练。

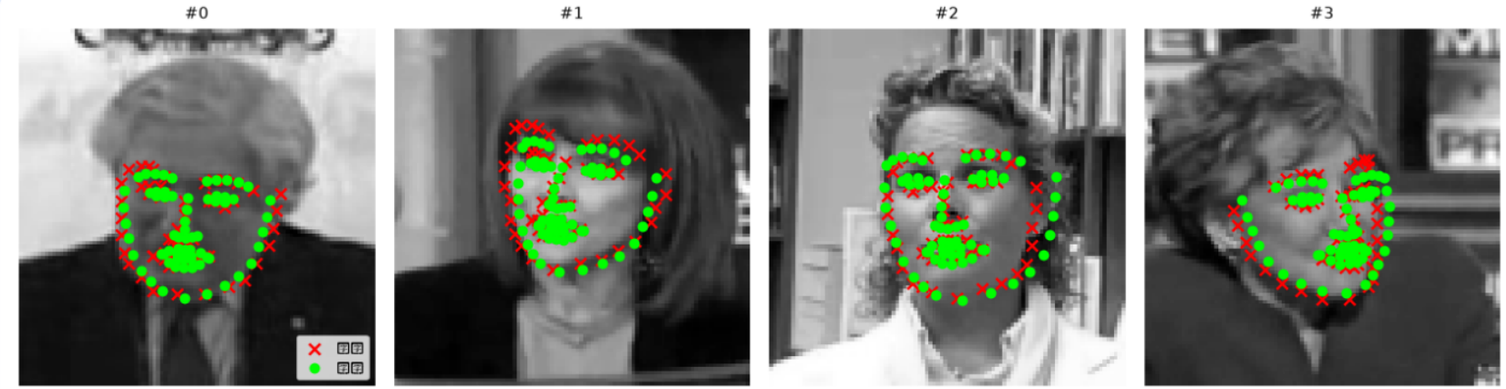


图 13·预测（绿点）vs 真实（红叉）：4 个验证样本上高度贴合五官轮廓

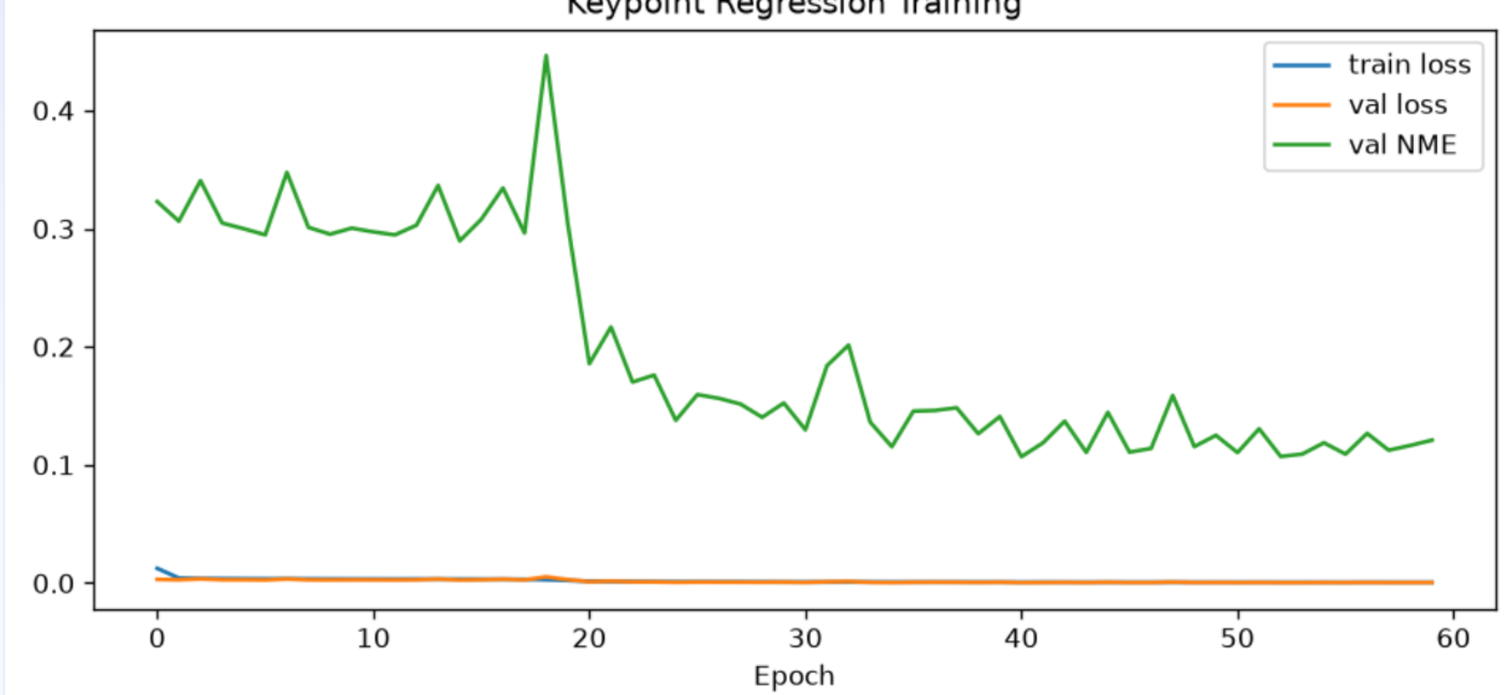


图 14·关键点训练/验证损失曲线（无过拟合迹象）

```
[Epoch 15/60] train=0.00337 val=0.00282 NME=0.3079
[Epoch 20/60] train=0.00213 val=0.00160 NME=0.2214 *
[Epoch 25/60] train=0.00111 val=0.00083 NME=0.1608 *
[Epoch 30/60] train=0.00093 val=0.00085 NME=0.1666
[Epoch 35/60] train=0.00079 val=0.00056 NME=0.1280 *
[Epoch 40/60] train=0.00073 val=0.00065 NME=0.1407
[最佳] NME = 0.1106 （首次完整运行）
# 修复可视化掩码 reshape 后复训最终模型：
[最佳] NME = 0.1069 （最终模型，越小越好，<0.05 算优秀）
[完成] 结果保存在 f:\deeplearn\projects\03_face_keypoints\results
```

4.3 FER2013 七类表情分类（EmotionCNN）

双卷积块×4 + 全局平均池化 + Dropout，共 **117 万参数**；数据严重不平衡（disgust 仅 496 张 vs happy 8065 张），配合 `ReduceLROnPlateau` 调度。

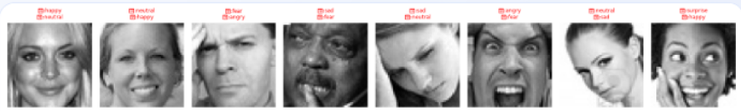
图 15·FER2013 验证集混淆矩阵（行=真实类别，列=预测类别，acc=65.1%）

	angry	disgust	fear	happy	neutral	sad	surprise
surprise	9		47	19	12	3	349
sad	68	3	70	36	136	267	5
neutral	41	1	38	45	418	68	7
happy	24		14	778	63	26	19
fear	61		235	21	67	68	37
disgust	11	29	4		1	5	1
angry	260	6	60	23	74	48	11

0 778 预测类别

```
[设备] cuda
[数据] 共 35887 条样本, 字段: ['img_bytes', 'labels']
[划分] train=32299  val=3588
[类别分布] {'angry': 4471, 'disgust': 496, 'fear': 4632, 'happy': 8065,
           'neutral': 5580, 'sad': 5492, 'surprise': 3563}
[模型] 参数量 1,175,399
[Epoch 01/25] loss=1.8058 val_acc=0.3342 *
[Epoch 03/25] loss=1.2823 val_acc=0.5460 *
...（完整逐轮日志已被终端清理，逐轮表现见下方误差样例与混淆矩阵）
最终验证准确率 acc = 0.651
```

结果解读：happy（778/886）、surprise（349/439）、disgust（29/52）识别最好；**angry / sad / fear / neutral 互相混淆严重**——这与类别不平衡和人类表情本身的语义重叠一致，是 FER2013 的公认难点[4]。



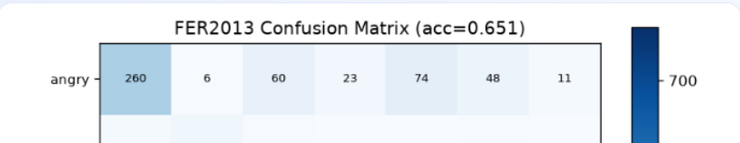


图 16·误分类样本展示

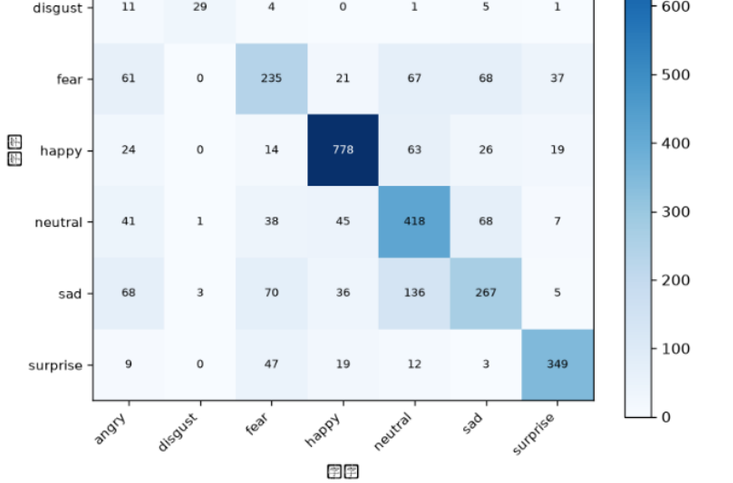


图 17·训练脚本输出的原始混淆矩阵

5. 项目四：CNN 特征图像检索

用 **ResNet18 去分类头的 512 维特征 + L2 归一化 + 余弦相似度**，在 CIFAR-10 万级图库上实现以图搜图。

5.1 系统设计

三步流程：① **建索引**——10000 张图库图过 ResNet18 提特征、L2 归一化后存 `gallery_features.npy`；② **检索**——查询图特征与图库做内积（归一化后内积=余弦相似度）取 top-k；③ **评估**——2000 次查询统计 Precision@1/5/10，与随机基线对比。

图 18·检索精度：CNN 特征 vs 随机基线（Precision@k, k=1/5/10）





指标	CNN 特征	随机基线	提升
Precision@1	74.8%	9.8%	7.6×
Precision@5	70.7%	10.0%	7.1×
Precision@10	68.4%	10.1%	6.8×



图 19·单次检索演示：查询图 + top-10 结果（绿框=同类，红框=异类）

```
[设备] cuda
[图库] 10000 张图, 特征维度 512
[查询] 第 42 张图, 真实类别 = dog
   top 1 dog          sim=0.844 ✓
   top 2 dog          sim=0.839 ✓
   top 3 dog          sim=0.839 ✓
   top 4 cat          sim=0.836 ✗
   top 5 dog          sim=0.836 ✓
   top 6 bird         sim=0.830 ✗
   ...
[评估] 计算中（2000 次查询）...
指标                    CNN特征        随机基线
Precision@1        74.8%       9.8%
Precision@5        70.7%       10.0%
Precision@10       68.4%       10.1%
[结论] CNN 特征的 Precision 远高于随机基线，证明深度特征有语义区分度
```

工程要点：相似度计算本质是矩阵-向量乘（`feats @ query`），万级图库毫秒级返回；扩展到百万级可引入 FAISS 等向量检索库做近似最近邻（ANN）。观察到 cat/dog/bird 相似度接近（0.82~0.84），说明**细粒度类别的特征空间距离天然更近**。

6. 总结：关键问题、修复与原理串联

6.1 调试过程中修复的关键 bug

项目	症状	根因	修复
01	转换后图片 FileNotFoundError	目标路径拼接到源图目录父级	为 `coco_to_yolo` 增加显式 `dst_img_dir` 参数
01	VOC 数据集无法训练	手头数据只有 image+mask 无 bbox	更换为 COCO 格式 football 检测数据集
02	conv_transpose2d 收到 2D 张量	生成器线性层输出未 reshape 4D	拆分 `fc` 与 `deconv`，`view(-1, 256, 7, 7)`
02	判别器卷核 4 > 输入 3×3	末层卷积配置错误	改 `kernel=3, padding=0`
02	DDPM sample() 输出异常	占位死代码未实现后验均值	补齐 `alphas_bar` 与去噪均值公式
03	关键点 y 最大 206 超出 96	**每张图尺寸不同**，坐标按各自原图标注	逐图动态缩放 `96/w, 96/h`
03	nme() 形状 (64) vs (128)	136 维掩码直接 reshape 成 68 分组错误	先 `reshape(-1, 68, 2)` 再 `.all(-1)`
03	loss 非标量无法 backward	MSELoss reduction='none'	掩码加权 `(diff*mb).sum()/mb.sum()`
04	KeyError: 'image'	CIFAR parquet 列名是 `img`	条件取列 `"img" if "img" in df.columns else "image"`

6.2 原理串联：一套方法论覆盖四类任务

① 迁移学习（项目1）：预训练骨干 + 小数据微调是工业界默认范式，本实验量化为 v1 +13.4 pp、v2 优化版（yolov8s）+28.4 pp mAP50（相对从零训练，v2 相对 v1 再 +15.0 pp）。**② 生成模型**（项目2）：GAN 以对抗博弈隐式建模分布，DDPM 以回归噪声显式建模逐步去噪——两者在同数据集上的质量/稳定性差异清晰可见。**③ 回归与评估设计**（项目3）：掩码损失处理缺失标注、NME 按双眼间距归一化使不同尺寸样本可比、混淆矩阵暴露类别不平衡。**④ 表征学习**（项目4）：分类网络的中间特征天然携带语义——**分类 / 检测 / 关键点 / 检索共享同一套卷积骨干与训练循环**，只是损失头不同。

对初学者最有价值的三条经验：数据正确性（坐标系/格式）优先于模型调参；对照实验（消融）比单一结果更有说服力；bug 修复过程本身就是对张量形状、数据管线理解最深的训练。

7. 附录：全部项目源代码

10 个实验脚本完整源码（含中文函数与注释），按项目与执行顺序排列。

项目一 · YOLOv8 目标检测

projects/01_yolov8_detection/01_prepare_dataset.py — 219 行

```
# -*- coding: utf-8 -*-
"""
项目1-①：数据准备 —— COCO 格式转 YOLO 格式
============================================
实验内容：
    1. 解压足球目标检测数据集（roboflow 导出，COCO 格式标注）
    2. 把 COCO 边界框 (x, y, w, h 绝对像素) 转成 YOLO 归一化格式
       YOLO txt 每行: <class_id> <cx/W> <cy/H> <w/W> <h/H>
    3. 生成 ultralytics 训练所需的 data.yaml 与目录结构
    4. 另存若干样例图片供 02_inference_demo.py 演示用

学到的原理：目标检测标注格式（COCO vs YOLO）、数据集划分
"""
import json
import os
import shutil
import sys
import zipfile

from PIL import Image
```

projects/01_yolov8_detection/02_inference_demo.py — 110 行

```
# -*- coding: utf-8 -*-
```

```
# -*- coding: utf-8 -*-
"""
项目1-②：YOLOv8 预训练模型推理演示（零样本检测）
================================================
实验内容：
    1. 加载 COCO 预训练的 yolov8n.pt，在本地样例图片上做推理
    2. 输出每个检测框的位置 (x, y, w, h)、类别、置信度
    3. 可视化检测结果并保存，直观理解「目标检测 = 定位 + 分类」

学到的原理：边界框、置信度、NMS 非极大值抑制、mAP 评估
"""
import glob
import os
import sys

from ultralytics import YOLO

# ---------------- 路径配置 ----------------
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
WEIGHTS = os.path.join(BASE_DIR, "weights", "yolov8n.pt")   # COCO 预训练权重
```

projects/01_yolov8_detection/03_fine_tune.py 133 行

```
# -*- coding: utf-8 -*-
"""
项目1-③：YOLOv8 目标检测训练实验（迁移学习 vs 从零训练）
============================================================
实验内容：
    A. transfer ：加载 COCO 预训练权重，在 VOC 子集上微调（迁移学习）
    B. scratch  ：相同结构、随机初始化，从零开始训练（对照组）
    两组使用完全相同的数据/超参数，对比收敛速度与最终 mAP

学到的原理：迁移学习、预训练特征复用、消融实验设计、mAP50/mAP50-95
"""
import argparse
import os
import sys
```

```
import time


from ultralytics import YOLO


# ---------------- 配置 ----------------
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

项目二 · GAN 与 Diffusion 生成

projects/02_gan_diffusion/01_dcgan_mnist.py 258 行

```
# -*- coding: utf-8 -*-
"""
项目2-①：DCGAN 生成手写数字（MNIST）
====================================
实验内容：
    1. 从本地 parquet 加载 MNIST 数据集（无需联网）
    2. 定义并训练 DCGAN（生成器 G + 判别器 D 对抗训练）
    3. 每个 epoch 保存生成样本网格图，观察生成质量演化
    4. 绘制 G/D 损失曲线，分析对抗训练的平衡过程

学到的原理：生成对抗网络、反卷积上采样、对抗损失、模式崩塌
"""
import io
import os
import sys
import argparse

import numpy as np
import pandas as pd
import torch
```

projects/02_gan_diffusion/02_ddpm_mnist.py 212 行

```
# -*- coding: utf-8 -*-
"""
```

```
项目2-②：DDPM 扩散模型生成手写数字（MNIST）
================================================
实验内容：
    1. 实现前向加噪过程（逐步把真实图片变成纯噪声）
    2. 训练一个轻量 UNet 预测每一步的噪声
    3. 反向去噪采样：从纯噪声一步步还原出数字
    4. 可视化「加噪链」与「去噪链」，直观理解扩散原理

学到的原理：扩散过程、噪声预测、马尔可夫链采样、余弦噪声调度
与 GAN 对比：扩散训练更稳定（回归损失 vs 对抗损失），但采样需要多步迭代、更慢
"""
import os
import sys

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
```

项目三 · 人脸关键点与表情识别

projects/03_face_keypoints/01_prepare_data.py　98 行

```
# -*- coding: utf-8 -*-
"""
项目3-①：人脸关键点数据准备
============================
实验内容：
    1. 读取原始数据（176x192 RGBA 人脸图 + 68 个关键点 CSV 标注）
    2. 统一转成 96x96 灰度图，等比例缩放关键点坐标
    3. 处理缺失值（丢弃含 NaN 的样本）
    4. 输出统一格式 npz：X(N,1,96,96) / y(N,136) / mask(N,136)

学到的原理：数据清洗、坐标与图像同步缩放、掩码处理缺失标注
"""
import os
```

```
import os
import sys

import numpy as np
import pandas as pd
from PIL import Image

# ---------------- 配置 ----------------
```

projects/03_face_keypoints/02_train_keypoints.py 221 行

```
# -*- coding: utf-8 -*-
"""
项目3-②：人脸关键点回归（定位五官）
====================================
实验内容：
    1. 加载统一格式的人脸数据（npz），68 个关键点 × (x, y) = 136 维输出
    2. 训练 CNN 回归网络直接预测 136 个坐标
    3. 用 NME（归一化平均误差）评估，比 MSE 更符合人眼感知
    4. 可视化预测关键点 vs 真实关键点，直观评估效果

学到的原理：回归任务、坐标归一化、NME 指标、热图法（对比讨论）
"""
import os
import sys

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt
```

projects/03_face_keypoints/03_fer2013_emotion.py 224 行

```
# -*- coding: utf-8 -*-
"""
项目3-③：人脸表情识别（FER2013 七类表情分类）
```

```
==========================================
实验内容：
    1. 加载 FER2013 表情数据（48x48 灰度人脸，7 类情绪）
    2. 训练 CNN 分类网络识别：angry/disgust/fear/happy/neutral/sad/surprise
    3. 输出整体准确率 + 混淆矩阵，分析哪些情绪容易混淆
    4. 可视化典型错误样本，理解困难分类的原因

学到的原理：分类任务、Softmax 交叉熵、混淆矩阵、类别不平衡（disgust 样本极少）
"""
import io
import os
import pickle
import sys

import numpy as np
import torch
import torch.nn as nn
```

项目四 · 图像检索

projects/04_image_retrieval/01_build_index.py 105 行

```
# -*- coding: utf-8 -*-
"""
项目4-①：构建图像检索索引（以图搜图）
========================================
实验内容：
    1. 加载 CIFAR-10 测试集（10 类共 10000 张小图）
    2. 用 torchvision 预训练 ResNet18 去掉分类头，提取 512 维特征向量
    3. L2 归一化后构建向量索引（余弦相似度 = 归一化后的内积）
    4. 保存特征矩阵与标签，供 02_search.py 检索使用

学到的原理：嵌入向量（embedding）、特征空间、余弦相似度、向量检索
"""
import io
import os
```

```
import sys

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
```

projects/04_image_retrieval/02_search.py 136 行

```
# -*- coding: utf-8 -*-
"""
项目4-②：以图搜图 —— 相似图片检索与评估
========================================
实验内容：
    1. 从图库中选一张查询图，检索最相似的 top-k 张图片并可视化
    2. 用类别标签量化评估：检索结果里同类占比 = Precision@k
    3. 批量测试全图库的 Precision@1 / @5 / @10，得到整体检索精度
    4. 对比实验：CNN 特征 vs 随机特征（体现深度学习特征的价值）

学到的原理：最近邻检索、Precision@k、特征表示质量评估
"""
import io
import os
import sys

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
```

参考来源

1. Ultralytics, YOLOv8 官方文档 — 训练/迁移学习指南。

https://docs.ultralytics.com/models/yolov8/

2. Ho et al., Denoising Diffusion Probabilistic Models (DDPM), NeurIPS 2020.
   https://arxiv.org/abs/2006.11239
3. Radford et al., Unsupervised Representation Learning with Deep Convolutional GANs (DCGAN), 2015.
   https://arxiv.org/abs/1511.06434
4. Goodfellow et al., Challenges in Representation Learning: FER2013 表情数据集, 2013.
   https://www.kaggle.com/datasets/msambare/fer2013
5. Krizhevsky, Learning Multiple Layers of Features from Tiny Images (CIFAR-10), 2009.
   Technical Report, University of Toronto

本报告由实验环境实测数据自动生成 · 2026-10-12 · 全部实验代码位于 `f:\deeplearn\projects\`